1st Place - GPT5.4, Gemini3.1, ClaudeOpus4.6 - KGMON Playbook!

Predict Customer Churn

Solution Writeup · 1st place · Apr 1, 2026

Wow, I'm excited to finish in 1st place! After shaking down in last month's playground competition, I worked extra hard this month to finish in top10. I probably worked too hard, but it was fun so it didn't feel like work :-D

KGMON Playbook for Tabular Data

This top solution follows the KGMON Playbook 2026 for tabular data: Blog here, Slides here, GitHub here, GTC Training Lab here, KGMON homepage here, Nvidia cuDF cuML homepage here.

- EDA

- Build Baselines
- GPU Feature Engineering
- GPU Hill Climbing
- Stacking
- Pseudo Labeling
- GPU Extra Training

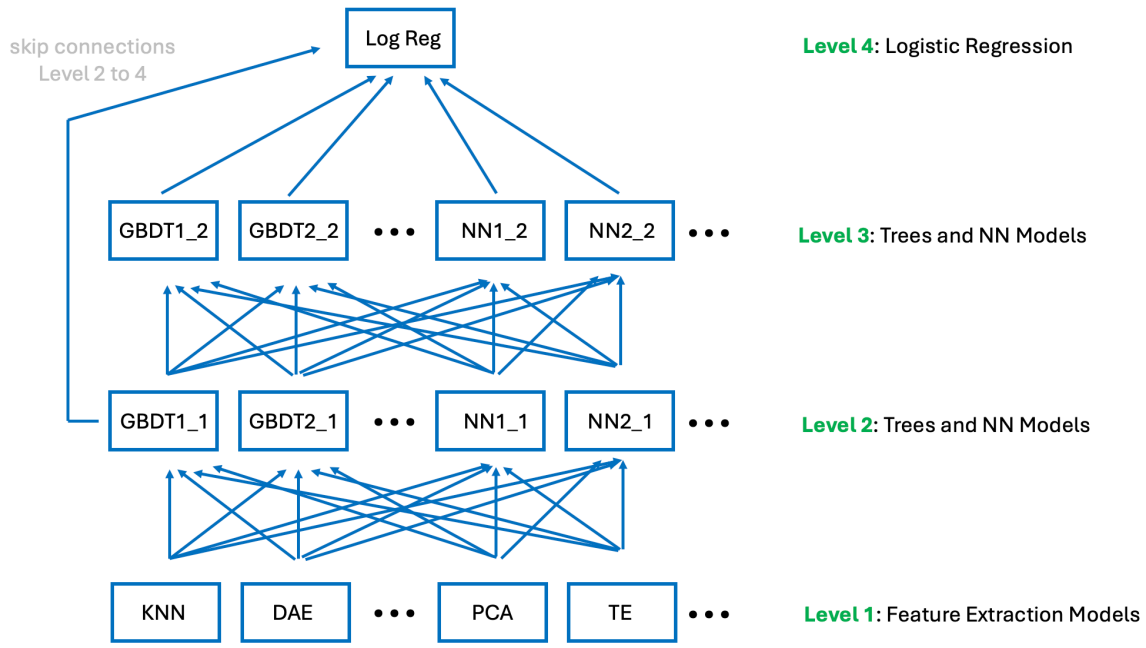
GPT5.4, Gemini3.1, ClaudeOpus4.6 and 4xA100 GPUs!

LLM tools are so exciting! All of this solution's code was written by GPT5.4, Gemini3.1, ClaudeOpus4.6 and experimentation was accelerated with GPU, Nvidia cuDF, and Nvidia cuML. These agents learned to understand and follow the KGMON Playbook for tabular data and finish 1st place!

In the month of March 2026, these LLMs wrote 600,000 lines of code! They built and trained 850 models on 4xA100 GPU and wrote and ran 50 EDA scripts! Wow unbelievable!

1. Final Solution - Four Level GBDT NN Stack

Our final solution is a 4 level stack. The first level is feature extraction models like Nvidia cuML KNN, PyTorch Denoising Auto encoder, PCA clustering, Nvidia cuML Target Encoding, etc etc. These models aggregate information from other rows to augment the current row. The second level is many GBDT and NN using 5x5 nested OOF from level 1. The third level is many GBDT and NN using 5x5 nested OOF from level 2. The fourth and final level is Nvidia cuML Logistic Regression using OOF from level 3 and level 2.



Besides the benefit of a large deep stack, the secret sauce involved creating 850 diverse potential models to select a final 150 models from. And the secret sauce involved creating 100s of diverse feature engineering techniques. Below we summarize our models and feature engineering. Many ideas were taken from public notebooks and references are provided at the end of the write up.

In addition to building models, GPT5.4, Gemini3.1, ClaudeOpus4.6 were prompted to perform their own EDA to determine the relationship between `train.csv 600k row synthetic data` and `7k original data` and `train.csv targets`. Then they used their discoveries to create many new feature engineering ideas to extract signal.

2. Feature Engineering

Feature engineering was the dominant driver of performance. Nearly all 150 models share a common core, with different models exploring additional specialized techniques.

2.1 Snap Features (universal)

Map each synthetic float to its nearest value in the original IBM dataset:

```
MC_snap      = nearest MonthlyCharges in original IBM
data
TC_snap      = nearest TotalCharges in original IBM
data
MC_snap_diff = MonthlyCharges - MC_snap   # synthetic
noise
TC_snap_diff = TotalCharges  - TC_snap
```

Used in nearly every model. The snap value recovers the "true" original feature; the diff encodes the generator's perturbation magnitude.

2.2 Digit and Decimal Extraction (~60 models)

The most pervasive technique. Extract the digit at every decimal position:

```
frac    = x - floor(x)
d1      = floor(frac * 10)         # 1st decimal
digit
d2      = floor(frac * 100) % 10   # 2nd decimal
digit
frac100 = round(frac * 100)        # 2-digit integer
representation
mod10   = floor(x) % 10
mod100  = floor(x) % 100
```

Applied to raw `MonthlyCharges`, `TotalCharges`, and their snap values. Also: fractional residuals from common denominators (1/2, 1/4, 1/5, 1/10), is-round flags (`frac < 0.005`), and digit pair combinations as categorical strings.

2.3 Target Encoding — Nested / Leak-Free (~90 models)

All target encoding (TE) uses a nested inner 5-fold loop within each outer CV fold to prevent label leakage:

- Applied to: all 16 raw categorical columns, bigram/trigram combos, binned numerics, anchor keys, and numeric × categorical snap products
- TE statistics beyond the mean: std, min, max, median, quantiles (5th, 10th, 45th, 55th, 90th, 95th)

- **Original IBM priors**: churn probability per feature value computed once from the 7,032-row original dataset — zero leakage, as the original data carries no synthetic training labels

2.4 Arithmetic Interactions (~45 models)

```
TC_deviation      = TotalCharges - tenure * MonthlyCharges
TC_snap_exp_dev   = TC_snap - tenure * MC_snap   # original billing anomaly
TC_per_month      = TotalCharges / (tenure + 1)
MC_to_TC_ratio    = MonthlyCharges / (TotalCharges + 1e-9)
```

`TC_deviation` is particularly powerful — it captures how much a customer's actual total charges deviate from what their monthly charge and tenure would predict.

2.5 Multi-Scale Binning (~52 models)

Quantile bins (up to 5,000 bins), fixed-width bins, log-scale bins, and integer floor — all converted to categorical strings and then target-encoded. Fine-grained quantile bins on `MonthlyCharges` (5,000 bins) essentially recover the original 1,584 unique values.

2.6 Categorical Cross-Features / Bigrams (~37 models)

```
df["bi_Contract_Internet"] = df["Contract"] + "__" + df["InternetService"]
```

Top-signal pairs: `Contract × InternetService`, `Contract × PaymentMethod`, `InternetService × PaymentMethod`. Trigrams extend this to 3-way combinations. Nested TE on these combos captures joint churn profiles unavailable to per-column TE.

2.7 Frequency / Count Encoding (~45 models)

```
freq = all_data[col].value_counts(normalize=True)
all_data[f"freq_{col}"] = all_data[col].map(freq)
```

Especially powerful for `MC_snap` (1,584 unique values): the frequency encodes how many synthetic rows cluster around each original IBM value, which correlates with the original customer's representativeness.

Count ratio features (`synthetic_count / original_count`) measure how much the generator oversampled each value.

2.8 Service Count Aggregations (~30 models)

```
svc_cols = ["OnlineSecurity","OnlineBackup","DeviceProtection",

"TechSupport","StreamingTV","StreamingMovies","MultipleLines"]
svc_yes = sum((df[c]=="Yes").astype(int) for c in svc_cols)
```

Also: `has_internet`, `has_phone`, per-service binary flags (`ISYES_`, `ISNO_`, `ISOTHER_`).

2.9 Original IBM Dataset Lookup (~7 models)

Build a `cKDTree` on the standardized `(MonthlyCharges, TotalCharges, tenure)` columns of the 7,032 original IBM rows. For each synthetic row, find the nearest original customer and attach their churn label as a feature. This provides a zero-leakage, noise-free "ground truth" anchor.

2.10 Radix Interaction Features (~15 models)

```
radix = int(MC_snap * 100) + cat_code * 100_000
```

Encodes a (continuous, categorical) pair as a single integer category. Very natural for tree models: a single split on this integer is equivalent to splitting on both the numeric value and the categorical value simultaneously.

2.11 Synthetic Artifact Detection (specialized)

Several models go further in exploiting the generator's fingerprints:

- **Fractional fingerprints**: `intlike_count` (frac < 0.001), `quarterlike_count`, `halflike_count` — how many of a row's numeric values look like round numbers

- **TF-IDF character n-grams**: Extracted from the *string representation* of `MonthlyCharges` and `TotalCharges` — detects repeating decimal patterns
- **Benford's Law deviation**: The leading-digit distribution of synthetic values deviates from Benford's Law; the deviation itself is a feature
- **Drift ratios**: `log1p(train_freq / orig_freq)` — measures how much the generator over/under-sampled each value

2.12 Projection / Manifold Features (~7 models)

Fit PCA (12 components) and Gaussian Random Projection (12 components) **on the original IBM data**, then project each synthetic row into that space. This gives the model explicit geometric information about how each synthetic customer relates to the true data manifold. Also: cyclical tenure features `sin/cos(tenure × 2π/12)` and `×2π/24`.

3. Tree Models

90 of the 150 ensemble models are tree-based, spanning five libraries. Each library grows

trees differently and benefits from different feature engineering strategies.

Library Comparison

Pr	XGBo	LightGBM	CatBoost	YDF	cuML RF
Tre	Depth-	Leaf-wise	Symmetric (obliviou	Depth-wise	Depth-wise
Ens	Boosting	Boosting	Boosting	Boosting	Bagging (Random Forest)

Cat	Manual	Manual TE; GPU	Native ordered target	Auto-binni	Manual TE required
Key	max_de	num_leaves, min_chi	l2_leaf_reg, random	max_depth,	n_estimators, max_features, min_sam

3.1 XGBoost — 37 Models

XGBoost accumulated the widest variety of feature engineering experiments:

- **Baseline ports**: Public Kaggle solutions, 5-fold nested TE on all 16 categorical columns
- **Anchor-based TE**: Use the `(MC_snap, tenure)` anchor key as the primary grouping column for TE — directly encodes which original IBM customer archetype each row belongs to
- **Self-supervised auxiliary predictions**: Train a separate model on the original 7,032 IBM rows to predict each feature column from all others; the `PRED_*` outputs encode consistency of each synthetic row with the original IBM distribution
- **Digit position features**: Extract d1–d4, fractional part, mod10, mod100 from all numeric columns; use `max_bin=16,000` for fine-grained splits on decimal artifacts
- **Comprehensive feature union**: Combine the best FE from all earlier XGB experiments into a single 200+ feature matrix
- **Bigram / trigram combos**: All categorical pairs/triples as string tokens, nested TE on each
- **GPU-accelerated pairwise TE**: 48 numeric × categorical pair features via cuML `TargetEncoder`
- **Multi-seed rank blending**: 3 random seeds, rank-transform before averaging for stable calibration
- **XGBRanker — pairwise ranking objective**:
  Uses `rank:pairwise` instead of `binary:logistic`, directly

optimizing the ordering that ROC AUC measures; Platt-scaled to [0,1]

3.2 LightGBM — 22 Models

LightGBM's leaf-wise growth finds deeper, more specialized patterns in the same feature space:

- **Baseline and public notebook port**: `num_leaves=63`, sklearn `TargetEncoder`
- **High-cardinality distribution features**: Count ratio features (`row_count / group_count`) capture synthetic generation frequency
- **XGB feature ideas ported to LGBM**: The same feature engineering applied to LightGBM produces genuinely different predictions due to leaf-wise vs. depth-wise growth
- **Optuna-optimized**: `num_leaves=77`, `learning_rate=0.00833`, `min_child_samples=56` — the low `num_leaves` prevents overfitting the noisy synthetic patterns
- **Synthetic artifact fingerprinting**: Prototype distance features to original IBM clusters; fractional rounding features
- **KDTree nearest-neighbor target feature**: cKDTree on original IBM numeric columns → nearest original customer's churn label as a feature

3.3 CatBoost — 22 Models

CatBoost's ordered target statistics (OTS) compute leak-free TE internally at each tree node — no manual TE loop is needed for raw categorical columns. This redirects FE effort to numeric interactions:

- **Minimal FE / native categoricals**: Pass all 16 categorical columns via `cat_features`, let CatBoost's OTS handle encoding — equivalent to leak-free TE, for free
- **Binned cross-term interactions**: Pre-compute numeric pair interactions (`a_bin × 9 + b_bin`) because CatBoost's symmetric (oblivious) trees apply the same split at every node of a

given depth and cannot chain splits to approximate interactions — they must be explicit input features

- **Borrowed RealMLP FE**: Cross-pollinate neural network FE ideas (digit features, 3-way combo tokens) into CatBoost
- **Optuna-optimized**: Tuned `random_strength=2.877` and `bagging_temperature=0.264` — CatBoost-specific regularizers with no XGBoost/LightGBM equivalents

3.4 YDF (Yggdrasil Decision Forests) — 2 Models

Google's YDF with `max_depth=2` produces ultra-shallow stumps — extremely regularized, smooth prediction surfaces that are almost orthogonal to the deep-tree models. Its value is purely ensemble diversity.

3.5 Nvidia cuML Random Forest — 2 Models

RAPIDS cuML's GPU-accelerated Random Forest provides the only **bagging** ensemble in the tree family — all other tree models use boosting. Because bagging averages many independently-grown deep trees rather than greedily correcting residuals, the cuML RF produces a fundamentally different prediction surface that adds genuine diversity to the boosted-tree bloc.

4. Deep Learning Models

60 of the 150 models are neural networks, spanning 25 distinct architecture families. All use PyTorch (directly or via `pytabkit` / `pytorch-tabular` / `pytorch-frame`) with CUDA acceleration.

4.1 Embedding MLP — Standard PyTorch (10 models)

The foundational DL architecture: learned categorical embeddings (dimension ≈ √cardinality) concatenated with raw numerics, passed through a 2–3 layer MLP (512→256→1) with dropout and BatchNorm/LayerNorm. Seven variants explore diverse training configurations

(batch size 256–8192, LR 2.5e-5–2e-3, cosine annealing, label smoothing ε=0.02) to maximize prediction diversity.

4.2 Feature Interaction MLP — Pair Embeddings (2 models)

Every ordered pair of the 19 input features gets its own embedding table (171 pairs, dim capped at 16), then all pair embeddings are concatenated and passed through a 3-layer MLP. Explicitly models every second-order feature interaction as a learnable embedding rather than relying on MLP hidden layers to discover them implicitly.

4.3 Enhanced Feature MLP — XGB-Inspired Transforms (4 models)

Same Embedding MLP backbone but numerical features are enriched with five transforms borrowed from gradient boosting practice: frequency encoding (FREQ), rank transform (RANK), log1p, square-root (SQRT), and reciprocal (INV1P), all standardized. Bridges the gap between tree-based and neural approaches by giving the MLP pre-processed numerical representations that mimic the decision boundaries trees naturally find.

4.4 RealMLP — pytabkit (9 models)

`RealMLP_TD_Classifier` from the `pytabkit` library. Key innovations: piecewise-linear representations (PLR) for numerical features (bin-then-learn approach giving quasi-tree-like inductive bias), SiLU activations, L2 normalization, robust scaling, label smoothing, flat+cosine LR schedule, and an 8-member internal ensemble baked into each fit (`n_ens=8`). One of the strongest individual architectures.

4.5 GraphSAGE GNN (4 models)

Each training/test row becomes a graph node; a KNN graph (k=8, built via cuML GPU-accelerated KNN) connects each customer to its nearest neighbors in feature space. Two SAGEConv layers aggregate neighbor information before an MLP classification head. Allows each customer's predicted churn probability to be informed by the churn patterns of their k nearest similar customers — a structural inductive bias unavailable to any MLP.

4.6 FT-Transformer (4 models)

Each input feature (numerical or categorical) is tokenized into a fixed-dimension embedding, then a Transformer encoder (multi-head self-attention + FFN) processes the sequence of feature tokens. A `[CLS]` token is used for classification. Self-attention among feature tokens allows every feature to attend to every other feature at each layer, learning arbitrary high-order interactions. Implemented via `pytorch-frame`'s `FTTransformer` with `LinearPeriodicEncoder` for numericals.

4.7 TabTransformer (5 models)

Applies Transformer self-attention exclusively to categorical feature embeddings, then concatenates context-aware categorical embeddings with layer-normalized numerics and passes through an MLP head. The hypothesis: categorical relationships (e.g., Contract type vs. InternetService) benefit most from attention, while numeric correlations are better left to the MLP.

4.8 TabM — Multiplicative Interactions (3 models)

`TabM_D_Classifier` from `pytabkit` with `tabm-mini-normal` architecture: PLR embeddings for numerics, MLP blocks with multiplicative (bilinear) interactions alongside additive paths, and `k=32` basis components linearly combined per sample. The implicit ensemble effect from k-component basis decomposition makes TabM one of the strongest individual DL models (OOF AUC **0.918788**).

4.9 TabICL — In-Context Learning (3 models)

`TabICLClassifier` from the `tabicl` library is a Transformer-based foundation model pre-trained on thousands of tabular datasets. At inference time it takes the training set as context and predicts test row labels by attending over training examples — zero fine-tuning required. The only model in the ensemble that requires no gradient updates at all.

4.10 GANDALF (2 model)

From `pytorch-tabular`: Gated Feature Learning Units (GFLU) gate inputs with learned sigmoid masks, select top-K most relevant features, and apply learned transformations across multiple stages. The multi-

stage gating progressively filters the feature set rather than relying on a monolithic weight matrix.

4.11 Self-Normalizing Network — SELU (2 model)

Deep MLP (256→128→64→32) with SELU activations and AlphaDropout, maintaining mean≈0 and variance≈1 at every layer without explicit normalization. Uniquely also treats numeric values as text: TF-IDF character n-gram features (128-dim) extracted from the string representations of `MonthlyCharges` and `TotalCharges`, plus Benford's Law likelihood features.

4.12 Tabular ResNet (2 model)

ResNet-style MLP (4 residual blocks, pre-activation BatchNorm→ReLU→Linear→Dropout→Linear + skip) combined with manifold projection features (PCA + GRP fitted on original IBM data) and cyclical tenure features (sin/cos with periods 12 and 24 months).

4.13 RFF Kernel Network (3 models)

Numerics are transformed via Random Fourier Features (RFF, dim=1,024) to approximate an RBF kernel, then concatenated with categorical embeddings and k=32 K-Means prototype distances fitted on original IBM data, and passed through an MLP. RFF projection explicitly maps inputs into a space where a linear classifier approximates a kernel SVM with RBF kernel.

4.14 Denoising Autoencoder (DAE) (10 models)

A PyTorch encoder-decoder (hidden 128→16 latent) trained exclusively on the 7,032-row original IBM dataset to reconstruct input features corrupted with Gaussian noise; reconstruction errors and latent embeddings computed on synthetic rows measure each row's deviation from the true IBM data manifold, which are then fed as features into XGBoost. The most powerful variant (dae-8907) combined DAE-derived features with a union of the strongest XGBoost feature sets, reaching **0.918345** — matching the best individual models in the ensemble.

4.15 Field-Aware Factorization Machine (FFM) (10 models)

Extends the standard FM by learning field-specific factor vectors: feature i has a separate embedding `v_{i→j}` for every target field j, so pairwise interactions are computed with context-aware representations rather than a single global vector per feature. Competitive at ~0.9159 without any deep MLP branch.

4.16 Deep Residual MLP (ResNet) (4 models)

A 4-block residual MLP (hidden=512, BatchNorm→GELU→Linear + skip connection) with categorical embeddings scaled as 1.6×√cardinality; uniquely augments each training fold with the original IBM rows so the model sees ground-truth customer archetypes directly alongside synthetic training examples. Reached 0.916376.

4.17 Factorization Machine (FM) (3 models)

A pure FM with no deep branch, modeling all pairwise feature interactions as `<v_i, v_j>·x_i·x_j` with factor dimension k=32 — the simplest non-linear model in the experiment suite beyond logistic regression, competitive at 0.9155 purely from second-order interactions with no hidden layers. Serves as a useful lower bound showing how much signal is captured by pairwise interactions alone.

4.18 DeepFM / DeepFFM (3 models)

Combines an FM or FFM interaction component with a parallel deep MLP branch sharing the same embeddings; the two paths (FM pairwise terms + MLP higher-order terms) add their logits at the output. The deep branch adds higher-order interactions beyond the second-order FM terms and slightly exceeds the pure FM (0.9154–0.9158).

4.19 Liquid Neural Networks (LNN) (3 models)

Inspired by the C. elegans connectome: each neuron has a learnable time-constant parameter `τ` controlling state-decay dynamics, giving the network an ODE-like continuous-time structure even on static tabular inputs. The embedding variant (lnn-4401, hidden=384, 4 layers, 0.9151) outperformed the plain variant (lnn-4400, 0.9128).

4.20 Variable Selection Network (VSN / TFT-style) (4 models)

Implements the Variable Selection Network from the Temporal Fusion Transformer (TFT) paper using Gated Linear Units (GLU) and Gated Residual Networks (GRN): each feature passes through a dedicated GRN, and outputs are recombined via learned softmax attention weights providing explicit feature importances. The best variant (G-10202) adds frequency snapping — collapsing rare continuous values to the nearest frequent value before embedding — to reach 0.9141, below most MLP variants of similar complexity.

4.21 TabNet (3 models)

Sequential sparse attention: at each decision step a learned mask selects a subset of features to process, conditioned on the aggregated representation from all prior steps, with the summed masks yielding interpretable feature importances. Both variants use self-supervised pre-training on augmented original IBM data before supervised fine-tuning via `pytorch-tabnet`'s `TabNetPretrainer` topping out at ~0.9122.

4.22 Trompt (PyTorch Frame) (3 models)

A prompt-based tabular model from the `torch_frame` library that maintains `num_prompts=8` parallel context vectors through `num_layers=4` processing stages, applying cross-entropy loss independently at every layer (layer-wise supervision) and averaging layer predictions at inference — adapting the prompting paradigm from NLP to tabular data. Reached **0.916291** with minimal feature engineering (no target encoding, no digit features) and only 6 training epochs, still improving at the final epoch.

4.23 DANet — Deep Abstract Network (1 model)

Within the FT-Transformer notebook: Abstract Layers select sparse subsets of input features via learnable attention masks, apply shared linear transforms to selected features, and stack multiple layers to progressively build higher-order representations. A differentiable feature-interaction filter inspired by tree splits.

4.24 TabPFN v2.6 Foundation Model (1 model)

A prior-data fitted Transformer trained offline on millions of synthetic tabular datasets that performs in-context learning at inference: the training set is passed as context and test row labels are predicted via

attention over training examples, requiring zero gradient updates on the target data. At this dataset scale (475k training rows), the model must subsample to 5k–10k rows per estimator due to context-length limits — with 32 estimators at 10k rows each outperforming 64 estimators at 5k rows (0.9134 vs. 0.9127) — capping performance well below what full-dataset models achieve.

4.25 DAE Transfer Learning (2 models)

Two-phase training: a denoising autoencoder (110→64→32, corrupting 30% of features with swap noise and Gaussian noise) is first pre-trained unsupervised on the combined train+test set, then a lightweight classification head is attached to the frozen encoder and fine-tuned with supervised labels — standard self-supervised transfer learning applied to tabular data. The 32-dimensional bottleneck learned a compressed representation of the full feature space, but the supervised fine-tuning phase could not exceed 0.915 despite the pre-training advantage.

5. Stacking and Ensembling

Meta-learner

All 150 base model OOF predictions are combined via a **Nvidia cuML L2-penalized Logistic Regression** (logit stacking) fit on the training labels. The L2 penalty prevents any single model from dominating the stack and naturally handles the high correlation among similar models.

Second-layer stacking

Several models (`_stk` suffix) are themselves trained on OOF predictions from earlier models as additional input features — creating a two-level hierarchy: raw feature models → stacked models → final logit meta-learner.

Diversity strategy

Diversity was intentional and systematic. The 150 models span:

- 4 GBDT libraries with fundamentally different tree structures
- 25 DL architecture families across attention, graph, kernel, multiplicative, gated, and foundation model paradigms

- 12+ distinct feature engineering pipelines (anchor TE, digit features, radix interactions, survival analysis, TF-IDF on numerics, KDTree lookup, pairwise embeddings, etc.)
- Multiple random seeds and Optuna-tuned hyperparameter configs

The logit stacker learns which model to trust for which region of the feature space.

Hill climbing for model selection

Models were added to the ensemble one at a time using greedy forward selection (hillclimbing) based on OOF AUC.

6. Hardware and Infrastructure

- **4× NVIDIA A100 80GB PCIe** (devices 0–3)
- **RAPIDS conda environment**: cuML for GPU-accelerated logistic regression, KNN, and target encoding
- **Training protocol**: All models use 5-fold `StratifiedKFold` with `SEED=42`
- **OOF / prediction naming convention**:
  - OOF: `oof_<description>_v{VER}.npy`
  - Test: `pred_<description>_v{VER}.npy`

7. Summary

The core of this solution is treating the synthetic data generation process as a source of structured signal rather than noise. The two most impactful techniques were:

1. **Snap features**: Mapping synthetic floats to their nearest original IBM value to recover the true customer archetype
2. **Decimal digit extraction**: Exploiting the artifacts left by the generator's rounding/sampling behavior as classification features

These signals were consumed by an unusually diverse ensemble — 37 XGBoost variants, 22 LightGBM variants, 22 CatBoost variants, 2 YDF models, 2 Nvidia cuML RF and 60 neural networks spanning 25

architectures — stacked via L2-penalized logistic regression for a final ensemble `OOF AUC of 0.91985`.

8. References — Public Notebooks Used as Starting Points

This section lists every public Kaggle notebook or discussion post referenced in the creation of this solution. Each reference was adapted into one or more local models.

Total unique references: 39

Kaggle Notebooks by Author

angelosmar1

https://www.kaggle.com/code/angelosmar1/s5e11-lgbm-cv-0-92813-2nd-place

LightGBM 2nd-place solution from Playground Series S5E11 achieving CV 0.92813; uses

nested target encoding and digit-position features as the core FE strategy.

Adapted in: `lgbm2-2700.ipynb`

azzamradman

https://www.kaggle.com/code/azzamradman/xgb-decaying-learning-rate-boost-guaranteed

XGBoost with a decaying learning rate schedule ("boost guaranteed") applied to the churn

dataset; notable for the LR decay trick rather than fixed learning rate.

Adapted in: `xgb10-3601.ipynb`

badalkrsharma

https://www.kaggle.com/code/badalkrsharma/cv-0-9163-xgb-lgb-multi-seed-ensemble

Multi-seed ensemble of XGBoost and LightGBM achieving CV 0.9163; averages predictions

across several random seeds to reduce variance.

Adapted in: `lgb-1000.ipynb`

https://www.kaggle.com/code/badalkrsharma/rank-blend-cv-0-91865-xgb-lgb-cat

Rank-blend of XGBoost, LightGBM, and CatBoost with pseudo-label augmentation, achieving

CV 0.91865; rank-transforms individual model predictions before blending for robust calibration.

Adapted in: `cat6-5700.ipynb`

https://www.kaggle.com/code/badalkrsharma/xgb-lgb-multi-seed-ensemble

Multi-seed XGBoost + LightGBM ensemble notebook (shared competition version of the

cv-0-9163 entry above).

Referenced in: `ensemble-10.ipynb`

blamerx

https://www.kaggle.com/code/blamerx/auc-0-91925-xgboost-bi-tri-target-encoding

XGBoost with bigram and trigram categorical cross-features, each target-encoded in a

leak-free nested CV loop; achieved AUC 0.91925 and was the source of the bi/tri-TE

feature engineering pipeline used across many ensemble models.

Adapted in: `RealMLP5-5500.ipynb`

https://www.kaggle.com/code/blamerx/s6e3-0-91902-optimized-catboost

Optuna-optimized CatBoost for S6E3 (CV 0.91902);
tunes `random_strength`,

`bagging_temperature`, and `l2_leaf_reg` with native categorical handling.

Adapted in: `cat-7300.ipynb`

https://www.kaggle.com/code/blamerx/s6e3-0-91906-optimized-lightgbm

Optuna-optimized LightGBM for S6E3 (CV 0.91906);
tunes `num_leaves`, `min_child_samples`,

and `learning_rate` alongside bigram/trigram target encoding.

Adapted in: `lgbm8-7200.ipynb`

https://www.kaggle.com/code/blamerx/s6e3-ridge-xgb-n-gram-0-91927-cv

Stacks a Ridge regression on character n-gram TF-IDF features (extracted from string

representations of numeric columns) with an XGBoost model, achieving CV 0.91927.

Adapted in: `ridge-xgb-9200.ipynb`

https://www.kaggle.com/code/blamerx/s6e3-tabm-advanced-features-0-91898-cv

TabM (`pytabkit`) with advanced feature engineering including bigrams, trigrams, digit

features, and piecewise-linear numeric embeddings; CV 0.91898.

Adapted in: `tabm-9100.ipynb`

cdeotte

https://www.kaggle.com/code/cdeotte/first-place-single-model-lb-38-81

1st-place single XGBoost model from a prior Kaggle Playground competition; the feature

engineering pipeline (digit extraction, nested TE, snap features) was ported as a

baseline starting point for this competition's XGBoost experiments.

Adapted in: `xgb3-1200.ipynb`

furqonaryadana

https://www.kaggle.com/code/furqonaryadana/lgbm-dart-woe-encoding-cpu-only

LightGBM with DART boosting and Weight-of-Evidence (WoE) encoding for categorical

features; WoE replaces the raw category code with the log-odds of the target given that

category, providing a calibrated monotonic encoding without requiring nested CV.

Adapted in: `lgbm-10100.ipynb`

greysky

https://www.kaggle.com/code/greysky/ps-s5e4-lgbm-cv-12-25-lb-12-15

LightGBM solution from Playground Series S5E4 (a regression task); the model structure

and hyperparameter choices were ported and retuned for binary churn classification.

Adapted in: `lgbm-8200.ipynb`

include4eto

https://www.kaggle.com/code/include4eto/realmlp-feature-engineering

RealMLP (`pytabkit`) paired with an extensive feature engineering pipeline including digit

features, frequency encoding, and target encoding.

Adapted in: `realmlp-8100.ipynb`

https://www.kaggle.com/code/include4eto/tabm-pseudo-labels

TabM (`pytabkit`) augmented with pseudo-label training: high-confidence test predictions

are added to the training set for a second training pass.

Referenced in: `ensemble-5.ipynb`

https://www.kaggle.com/code/include4eto/tabtransfomer-chatgpt-vibe-coding

TabTransformer implementation assembled with ChatGPT assistance ("vibe coding"); applies

self-attention to categorical embeddings, then concatenates with normalized numerics for

an MLP classification head.

Adapted in: `tabtran-8400.ipynb`

johnnyhyland

https://www.kaggle.com/code/johnnyhyland/s6e3-reframing-as-bayesian-survival-pymc

Reframes the churn binary classification task as a Bayesian survival analysis problem

using PyMC; models time-to-churn with a Cox-proportional-hazards-inspired likelihood,

then uses the survival probability at the observed tenure as the churn prediction.

Adapted in: `cox-8000.ipynb`

lightningv08

https://www.kaggle.com/code/lightningv08/s6e3-cv-0-91849-xgb-kfold-fe-pl

XGBoost with k-fold feature engineering and pseudo-label augmentation achieving CV 0.91849;

uses fold-safe target encoding computed within the CV loop before pseudo-label retraining.

Referenced in: `ensemble-4.ipynb`

mahoganybuttstrings

https://www.kaggle.com/code/mahoganybuttstrings/pg-s5e10-realmlp-cv-0-055936-lb-0-05549

RealMLP (`pytabkit`) solution from Playground Series S5E10; ported as the structural

template for RealMLP experiments on the churn dataset.

Adapted in: `RealMLP3-2300.ipynb`

https://www.kaggle.com/code/mahoganybuttstrings/pg-s5e11-xgb-cv-0-92818-pb-0-92923

XGBoost solution from PS S5E11 (CV 0.92818, public LB 0.92923); uses extensive pairwise

categorical target encoding and numeric rounding features as the primary FE approach.

Adapted in: `mahog-700.ipynb`

https://www.kaggle.com/code/mahoganybuttstrings/pg-s5e8-single-xgb-cv-0-975782-lb-0-97681

Single XGBoost model from PS S5E8 achieving CV 0.975782; the general XGBoost structure

and TE pipeline were ported for use on churn data.

Adapted in: `xgb6-2000.ipynb`

https://www.kaggle.com/code/mahoganybuttstrings/pg-s5e8-tabm-cv-0-976810-pb-0-97750

TabM (`pytabkit`) solution from PS S5E8 achieving CV 0.97681; uses piecewise-linear

numeric embeddings and multiplicative interactions.

Adapted in: `tabM-800.ipynb`

https://www.kaggle.com/code/mahoganybuttstrings/pg-s6e1-realmlp-cv-8-58748-lb-8-58006

RealMLP solution from PS S6E1; served as the structural template for the RealMLP ensemble

notebook combining predictions across multiple folds and seeds.

Adapted in: `RealMLP-600-ENSEMBLE.ipynb`

masayakawamata

https://www.kaggle.com/code/masayakawamata/s5e11-single-lgbm-tuned

Single tuned LightGBM from PS S5E11 with target encoding and digit features; adapted as

an LightGBM baseline with this competition's feature engineering added on top.

Adapted in: `lgb6-3800.ipynb`

https://www.kaggle.com/code/masayakawamata/s5e5-resmlp-cv0-05990

Residual MLP (ResMLP) architecture implemented in PyTorch for PS S5E5; the skip-connection

MLP design was ported as the backbone for the ResMLP neural network experiments.

Adapted in: `nn2-1602.ipynb`

mikhailnaumov

https://www.kaggle.com/code/mikhailnaumov/customer-churn-ensemble

Public ensemble notebook combining multiple churn model predictions for this competition;

referenced for ensemble construction methodology.

Referenced in: `ensemble-10.ipynb`

nalgirayergn

https://www.kaggle.com/code/nalgirayergn/xgb-predict-customer-churn-cv-0-91819

XGBoost model for customer churn achieving CV 0.91819; uses snap features (mapping

synthetic floats to nearest original IBM values) and digit extraction as its core FE.

Adapted in: `xgb10-3600.ipynb`, `xgb10-3601.ipynb`

omidbaghchehsaraei

https://www.kaggle.com/code/omidbaghchehsaraei/tabtransformer-cv-0-35327-lb-0-36542

TabTransformer baseline applying Transformer self-attention to categorical embeddings

before an MLP head; adapted as the starting architecture for TabTransformer experiments.

Adapted in: `nn2-1603.ipynb`

https://www.kaggle.com/code/omidbaghchehsaraei/the-best-solo-model-so-far-realmlp-lb-0-95397

Best single-model RealMLP submission in the competition at the time of reference

(LB 0.95397); used as a strong RealMLP baseline with grouped original-dataset stats as

the key feature engineering innovation.

Adapted in: `realmlp8-6300.ipynb`

ozermehmet

https://www.kaggle.com/code/ozermehmet/original-data-fe-single-xgb-cv-0-919-lb-0-916

Single XGBoost model (CV 0.919, LB 0.916) that uses original IBM dataset features as

engineered inputs — a precursor to the cKDTree nearest-neighbor lookup approach.

Adapted in: `xgb-7900.ipynb`

rawashishsin

https://www.kaggle.com/code/rawashishsin/s6e3-single-realmlp-lb-0-91633

Single RealMLP entry for this competition (LB 0.91633); uses bigram/trigram target

encoding alongside standard digit and snap features fed into `RealMLP_TD_Classifier`.

Adapted in: `realmlp-10800.ipynb`

siukeitin

https://www.kaggle.com/competitions/playground-series-s6e3/discussion/679983

Competition discussion post sharing a YDF (Yggdrasil Decision Forests) solution using

Google's decision forest library; the ultra-shallow `max_depth=2` configuration produces

highly regularized, smooth predictions valuable for ensemble diversity.

Adapted in: `ydf-3000.ipynb`

tsunamazda

https://www.kaggle.com/code/tsunamazda/predict-customer-churn-single-lightgbm

Single LightGBM model specifically targeting this churn competition; served as a clean,

minimal LightGBM baseline before more complex feature engineering was layered on.

Adapted in: `lgbm-5-3700.ipynb`

yekenot

https://www.kaggle.com/code/yekenot/ps-s5-e11-fttransformer-pytorch-frame

FT-Transformer implemented via the `torch_frame` (PyTorch Frame) library for PS S5E11;

tokenizes each feature into a fixed-dimension embedding and applies Transformer

self-attention across the feature sequence before a classification head.

Adapted in: `fft-4300.ipynb`

https://www.kaggle.com/code/yekenot/ps-s5-e5-liquid-nn-pytorch

Liquid Neural Network implemented from scratch in PyTorch for PS S5E5; each neuron has a

learnable time-constant `τ` giving ODE-like continuous-time dynamics applied to tabular

inputs, inspired by the C. elegans connectome.

Adapted in: `lnn-4400.ipynb`

https://www.kaggle.com/code/yekenot/ps-s6-e3-realmlp-pytabkit

RealMLP via `pytabkit` specifically targeting this S6E3 competition; uses piecewise-linear

numeric embeddings, SiLU activations, and an internal 8-member ensemble.

Adapted in: `cat-9300.ipynb`

https://www.kaggle.com/code/yekenot/ps-s6-e3-trompt-pytorch-frame

Trompt (Tabular Prompt model) via `torch_frame` for S6E3; maintains parallel prompt

vectors through multiple processing layers with layer-wise supervised loss at each stage.

Adapted in: `tromp-4200.ipynb`

yunsuxiaozi

https://www.kaggle.com/competitions/playground-series-s6e3/discussion/681229

Competition discussion post on TabICL — a Transformer foundation model pre-trained on

thousands of tabular datasets that performs in-context learning at inference time with

zero fine-tuning on the target dataset.

Adapted in: `tab-9500.ipynb`

https://www.kaggle.com/code/yunsuxiaozi/realmlp-from-scratchcv-0-91908/ RealMLP written from scratch specifically targeting this S6E3 competition; uses RobustScaleSmoothClipTransform, CategoricalFeatureLayer, ScalingLayer, NTPLinear, PBLDEmbedding.

Summary of References by Model Type

Model Type	References
XGBoost	azzamradman, badalkrsharma (×2), blamerx (×2), cdeotte, lightningv08, mahoganybuttstrings (×2), nalgirayergn, ozermehmet
LightGBM	angelosmar1, badalkrsharma, furqonaryadana, greysky, masayakawamata, tsunamazda
CatBoost	blamerx
RealMLP	include4eto, mahoganybuttstrings (×3), omidbaghchehsaraei, rawashishsin, yekenot, yunsuxiaozi
TabM	blamerx, include4eto, mahoganybuttstrings
TabTransformer	include4eto, omidbaghchehsaraei
FT-Transform	yekenot
Liquid NN	yekenot
Trompt	yekenot
TabICL	yunsuxiaozi
YDF	siukeitin

Bayesian Survival	johnnyhyland
Ridge + n-gram	blamerx
ResMLP	masayakawamata

9. Solution Code

I published my final level 4 `Nvidia cuML Logistic Regression` model here and 154 strong OOF and Test Preds in Kaggle Dataset here. The Logistic Regression model receives input from level 2 and level 3 models. In the list of input models, all models with suffix `"_stk"` are level 3 and the others are level 2.

Enjoy!